10 • 综合项目（server-health-check）

本册是阶段 7 的正文，从 7-01 起步；配套与路线见《17-关卡总表》。

本册共 8 关（7-01 ~ 7-08）。目标：交付王牌项目 v1 —— 本机巡检 + 阈值告警 + 报告与历史对比 + 容器化 + 指标进监控 + README/设计决策/讲述稿。

这是实战里最看重的那道题的主料，做完要能在干净环境按 README 跑通。

本册是什么：综合项目 —— 把前面所有零件拼成 server-health-check：巡检、告警、报告、容器化、接监控。 用法：八关走完，产出一个陌生人 clone 下来能跑通的作品（含 README 与讲述稿）。

动手做实验时照这两条来：实验闭环（先记录原状态 → 操作 → 按输出 / 退出码验证 → 恢复现场）和踩坑记录位（现象 → 原因 → 解决 → 一句话总结）——不必逐关都写，做完整册回头总结一次就够。

7-01 项目立项：巡检到底要检查什么

这一册是你的王牌项目：`server-health-check` —— 实战里最看重的"讲一件你真正动手做过的事"，讲的就是它。第一关先把"要检查什么、怎么算不正常"定下来，**不写代码**。

步骤 1：先回答一个问题

```
如果半夜收到告警"某台服务器好像不对劲"，你希望脚本替你回答哪几个问题？

1) 磁盘是不是快满了？      → df -h          （门槛：>80% 告警）
2) 内存是不是被吃光了？    → free -m        （门槛：可用 <10%）
3) CPU 负载是不是过高？    → uptime         （门槛：1 分钟负载 > 核数 × 1.5）
4) 关键服务还活着吗？      → systemctl is-active    （门槛：不是 active 就报）
—— 这四条就是版本 1 的检查清单（和阶段 3 的 v0 相比，多了"服务检查"）
```

步骤 2：给每条定"三件事"

检查项	命令	正常 / 异常门槛	异常时说什么
磁盘	`df -h /`	使用率 ≥ 80%	"根分区用量 87%，先查是谁在涨"
内存	`free -m`	可用 ≤ 10%	"可用内存只剩 380MB"
负载	`uptime` / `nproc`	1 分钟负载 > 核数 × 1.5	"负载 7.2，超过 4 核的 1.5 倍"
服务	`systemctl is-active ssh`	不是 `active`	"ssh 服务状态：inactive"

解读：**每一个检查都要有"门槛"和"异常时的人话"** —— 这就是"运维脚本"和"练习脚本"的区别：前者是给人看的、能直接进告警的。

步骤 3：写一页《检查清单 + 判定标准》（产出）

```
# server-health-check 检查清单 v1
| 检查项 | 命令 | 门槛 | 异常时输出 |
|-------|------|------|-----------|
| 磁盘   | df -h / | ≥80% | [告警] 根分区 87% |
| 内存   | free -m | 可用 ≤10% | [告警] 可用内存 380MB |
| 负载   | uptime  | >核数×1.5 | [告警] 负载 7.2（4 核）|
| 服务   | systemctl is-active | ≠active | [告警] ssh inactive |
退出码约定：0=全部正常，1=脚本自身出错，2=存在告警
```

完成判定

□ 有一页检查清单（四项 + 门槛 + 异常话术）　□ 能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇）　□ 能说出为什么每项都要有门槛　□ 能说出退出码 0/1/2 各自代表什么　□ 能说出这一版和 3-17 的 v0 差在哪（多了服务检查、门槛更明确）

回滚方式：纯文档，无风险。

实战价值：别人问"你这个巡检项目都查什么"，你能一条条说出"查什么、什么算不正常、为什么定这个门槛"——这就是"设计过"和"抄过"的区别。

7-02 v1：把四个检查写进脚本（subprocess + 阈值告警）

步骤 1：核心骨架（可直接复制，先跑通再改）

```
#!/usr/bin/env python3
"""server-health-check v1：四项本机巡检，超阈值告警"""
import subprocess
import sys


def run(cmd):
    """跑一条命令，返回输出；命令不存在/失败都抛异常给上层"""
    result = subprocess.run(cmd, capture_output=True, text=True, timeout=10)
    if result.returncode != 0:
        raise RuntimeError(f"{' '.join(cmd)} 失败：{result.stderr.strip()}")
    return result.stdout


def check_disk(threshold=80):
    out = run(["df", "-h", "/"])                        # 第二行第二列是使用率
    usage = int(out.splitlines()[1].split()[4].rstrip("%"))
    return usage >= threshold, f"根分区使用率 {usage}%"


def check_memory(min_free_pct=10):
    out = run(["free", "-m"])                           # Mem 那行：total / available
    parts = out.splitlines()[1].split()
    total, available = int(parts[1]), int(parts[6])
    pct = available / total * 100
    return pct <= min_free_pct, f"可用内存 {available}MB（{pct:.0f}%）"


def check_load(factor=1.5):
    cores = int(run(["nproc"]).strip())
    load1 = float(run(["uptime"]).split("load average:")[1].split(",")[0])
    return load1 > cores * factor, f"1 分钟负载 {load1}（{cores} 核，门槛 {cores * factor}）"


def check_service(name="ssh"):
    # 注意：is-active 用"返回码"表达状态（不在跑=3、不存在=4），
    # 所以这里不能用会抛异常的 run()，要自己接住返回码
    result = subprocess.run(["systemctl", "is-active", name],
                            capture_output=True, text=True, timeout=10)
    out = result.stdout.strip() or "（无输出）"
    return out != "active", f"服务 {name} 状态：{out}"


CHECKS = [("磁盘", check_disk), ("内存", check_memory), ("负载", check_load), ("服务",
check_service)]


def main():
    alerts = []
    for name, fn in CHECKS:
        try:
            bad, message = fn()
```

```
        except Exception as exc:
            print(f"[错误] {name} 检查失败：{exc}")
            return 1
        print(f"[{'告警' if bad else '正常'}] {name}：{message}")
        if bad:
            alerts.append(f"{name}：{message}")
    if alerts:
        print(f"\n共 {len(alerts)} 项异常")
        return 2
    print("\n全部正常")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

解读（四处值得说的设计）：① **每个检查是一个函数** —— 以后加"GPU 检查"只是再加一个函数；② **命令失败抛异常、上层统一处理**（不会静默算错）；③ **每项都打印"人话"**，不只是数字；④ **退出码 0/1/2**，可以被 crontab 或别的脚本判断。

步骤 2：跑两次 —— 正常态和告警态

```
$ chmod +x server_check.py && ./server_check.py
[正常] 磁盘：根分区使用率 42%
[正常] 内存：可用内存 3120MB（78%）
[正常] 负载：1 分钟负载 0.31（4 核，门槛 6.0）
[正常] 服务：服务 ssh 状态：active

全部正常
（两种常见情况：① 本机没装 ssh—报 Unit ssh.service could not be found.—把 check_service 的默认名换成你机器上一定有的服务（如 cron），或先装 openssh-server；② 装了 ssh 但显示 inactive—这是 26.04 的正常现象：新版 ssh 由 ssh.socket 按需拉起，"服务没在跑"≠机器有问题；要盯 ssh 就改查 ssh.socket，或干脆默认用 cron 这类常驻服务，免得把正常机器误报成告警。示例里 ssh 显示 active 是演示机的样子，以你机器为准。）
$ echo $?
0

# 把门槛调极端，看告警态
$ sed -i 's/threshold=80/threshold=1/' server_check.py && ./server_check.py
[告警] 磁盘：根分区使用率 42%
...
共 1 项异常
$ echo $?
2
```

步骤 3：故意造一个故障场景（这步最值钱）

```
# 造一个大文件把磁盘顶上去
$ fallocate -l 3G ~/lab/bigfile && ./server_check.py
[告警] 磁盘：根分区使用率 81%          ← 脚本真的抓到了（示例："3G 顶到 81%"是小沙箱盘的效果；你的盘大就用步骤 2 的 sed 阈值法，或按盘容量再造大文件）
$ rm ~/lab/bigfile && ./server_check.py   # 清理后回到正常

# 停一个服务看它抓不抓
$ sudo systemctl stop ssh.socket 2>/dev/null || sudo systemctl stop ssh
$ ./server_check.py | grep 服务
[告警] 服务：服务 ssh 状态：inactive
$ sudo systemctl start ssh        # 记得恢复！
```

回滚方式：实验文件 `rm ~/lab/bigfile`；停掉的服务 `systemctl start` 恢复。**动服务前先确认它是不是你正在用的（ssh 停了你自己会掉线，所以先开一个备用终端）。**

完成判定

□ 脚本四项检查都能跑出结果　□ 造过一次真实告警（大文件顶磁盘）并恢复　□ 退出码 0/1/2 都验证过　□ 能说清为什么每项检查用函数封装　□ 能说清 `subprocess.run` 的 `capture_output` / `text` / `timeout` 各干什么

实战价值：这是实战里最看重的那道题的主料 —— 你能讲"我怎么设计这个脚本、怎么造故障验证它、顺手处理了哪些边界"。有设计取舍的故事，才有说服力。

7-03 输出报告与历史对比

为什么要"留档"：只看一眼数字，你判断不了"这台机器是今天不对，还是一直都不对"。**把每次巡检写成文件，才有"趋势"**——这也是运维和临时工的区别。

步骤 1：把结果写成文件（带时间戳）

```
import datetime, os, pathlib

REPORT_DIR = pathlib.Path(os.environ.get("REPORT_DIR", pathlib.Path.home() / "lab" / "health-reports"))  # 允许用环境变量指定（容器里挂载就靠它）


def write_report(lines, alerts):
    REPORT_DIR.mkdir(parents=True, exist_ok=True)
    now = datetime.datetime.now()
    path = REPORT_DIR / f"{now:%Y%m%d-%H%M%S}.txt"          # 每次一个文件，按时间命名
    body = "\n".join([f"巡检时间：{now:%Y-%m-%d %H:%M:%S}", f"告警数：{len(alerts)}", *lines])
    path.write_text(body, encoding="utf-8")
    print(f"报告已写入：{path}")
    return path
```

解读：文件名带时间戳，**天然就是历史记录**；目录用 `mkdir(parents=True, exist_ok=True)` 保证第一次跑也不会因为目录不存在而失败。

步骤 2：和上一次比（这一步才是"运维视角"）

```
def last_reports(n=2):
    files = sorted(REPORT_DIR.glob("*.txt"))
    return files[-n:]


def diff_with_last(current_line):
    """从最近两次报告里，把磁盘那行挑出来对比"""
    files = last_reports(2)
    if len(files) < 2:
        return "（还没有上一份报告，明天再比）"
    def disk_line(path):
        for line in path.read_text(encoding="utf-8").splitlines():
            if "根分区使用率" in line:
                return line
        return ""
    return f"上次：{disk_line(files[0])}\n本次：{disk_line(files[1])}"


print(diff_with_last(""))
# 两次运行后输出示例：
# 上次：[正常] 磁盘：根分区使用率 42%
# 本次：[告警] 磁盘：根分区使用率 81%     ← 一眼看出涨了
```

解读：要点不在算法，而在**"先有历史、才有趋势"这个习惯**。生产上这叫"基线"（baseline）：没有基线，"这个数字正不正常"就只能靠猜。

步骤 3：接到定时任务上（让它自己跑）

```
$ crontab -e
0 * * * * /usr/bin/python3 /home/你的真实用户名/lab/server_check.py >> /home/你的真实用户名/lab/health.log 2>&1

# 一小时后确认：
$ tail -3 ~/lab/health.log
$ ls -1 ~/lab/health-reports | tail -3
20261009-231001.txt
20261010-001001.txt
20261010-011001.txt      ← 每小时一份，历史自动累积
```

行里两处 /home/你的真实用户名 记得换成你的真实路径；cron 不展开 ~，写绝对路径最稳。

回滚方式：`crontab -e` 删掉那一行；报告目录 `rm -rf ~/lab/health-reports`（先确认里面没有你要留的）。

完成判定

□ 每次运行都生成一份带时间戳的报告文件　□ 至少跑两次，能打印出"上次 vs 本次"的对比　□ 挂上 crontab，等它自己跑出至少两份　□ 能说出"基线"是什么、为什么没有基线就没法判断异常

实战价值：工作里常问"你怎么判断一台机器状态不对"，答"我留了历史报告，看的是趋势不是瞬间值"——这句话直接体现运维思维。

7-04 容器化：把巡检脚本做成镜像

拉不动基础镜像？（报 `i/o timeout`）按 **07 册的镜像加速器说明**配一次即可。

步骤 1：Dockerfile（照 4-05 的模板改）

```
FROM python:3.14-slim
RUN apt-get update && apt-get install -y procps && rm -rf /var/lib/apt/lists/*
WORKDIR /opt/check
COPY server_check.py ./
RUN chmod +x server_check.py
ENTRYPOINT ["python3", "server_check.py"]
```

（**为什么要装 procps**：slim 镜像里连 `free` / `uptime` 都没有——这是 4-01 学的"底座只给你最小集合"，缺什么自己装。）

解读：用 `python:3.14-slim` 当底座（比 ubuntu 小、又自带 Python）；**不用再 apt 装 Python** —— 这就是选底座的价值。

为什么用 ENTRYPOINT 而不是 CMD：镜像名后面跟的参数会**追加**到 ENTRYPOINT；写成 CMD 会把它**整条替换**（后面 `--skip-service` 那一步能生效就靠这个）。

构建卡在 apt-get？（国内网络常见）：容器的 apt 走 Debian 官方源，慢网下可能十几分钟拉不动。把步骤 1 的 RUN 换成下面这行（换源段会打印实际改了哪个源文件）再构建，几秒就能装完：`RUN for f in /etc/apt/sources.list.d/debian.sources /etc/apt/sources.list; do [ -f "$f" ] && echo 换源：$f && sed -i 's|deb.debian.org|mirrors.tuna.tsinghua.edu.cn|g' "$f"; done; apt-get update && apt-get install -y procps && rm -rf /var/lib/apt/lists/*`。换的只是下载通道、装的包不变；**注意：07 册配的加速器只管拉 Docker 镜像，管不到容器里的 apt**，两件事分开配。

步骤 2：构建、运行、看结果

```
$ docker build -t health-check:1.0 .
$ docker run --rm health-check:1.0
[正常] 磁盘：根分区使用率 1%          ← 容器自己的盘（数值随机器）
[正常] 内存：可用内存 8192MB（85%）     ← 装了 procps 才有 free
[正常] 负载：1 分钟负载 0.10（2 核，门槛 3.0）
[错误] 服务 检查失败：[Errno 2] No such file or directory: 'systemctl'   ← 容器里没有 systemd

$ echo $?
1
```

解读：**这是一个真实且重要的坑**：容器里通常没有 systemd，所以"检查服务"这条在容器内做不了。**更要紧的是**：这条一报错，`main()` 就 `return 1`——**后面的 `write_report` 根本不会执行、报告也就写不出来**（这正是下一步先用上开关的原因）。**三种正经处理办法**：① 把服务检查做成可选（`--skip-service` 参数，**下一段就给代码**）；② 服务检查改用"端口探活"（容器里 `nc -z localhost 22` 也能判断）；③ 这个检查项留给"在宿主机上跑"的场景，容器版只做前三项。

三种都行，**关键是你要能说清为什么**——这就是"设计决策"（下一关有模板）。

本册就按办法①做，给 `main()` 开头加两行：

```
def main():
    if "--skip-service" in sys.argv:                    # 容器里没有 systemd 时跳过服务检查
        global CHECKS
        CHECKS = [c for c in CHECKS if c[0] != "服务"]
    alerts = []
    ...（后面不变）
```

步骤 3：把宿主机目录挂进去（报告要落地）

跑之前先建好目录：`mkdir -p ~/lab/health-reports`——目录不存在时 Docker 会自动帮你建一个，但**属主是 root**，之后你自己写、删都麻烦。

```
$ docker build -t health-check:1.0 .        # 改完脚本要重新构建
$ docker run --rm -v ~/lab/health-reports:/reports -e REPORT_DIR=/reports health-check:1.0 --skip-service
$ ls ~/lab/health-reports | tail -2       # 报告确实落在宿主机上
```

解读：**容器是"用完就删"的，报告必须写到挂载出来的目录里**（4-03 学过"无状态的放容器里、有状态的挂出来"，这里就是它的真实应用）。

报告文件名的时间比墙上的钟慢 8 小时？正常——**容器默认走 UTC**（比北京时间慢 8 小时）；"上次 vs 本次"对比不受影响，只是历史文件排序观感怪。想让时间戳和宿主机一致：跑的时候加一个环境变量 `-e TZ=Asia/Shanghai` 即可——完整命令：`docker run --rm -e TZ=Asia/Shanghai -v ~/lab/health-reports:/reports -e REPORT_DIR=/reports health-check:1.0 --skip-service`。

回滚方式

`docker rmi health-check:1.0` 删镜像；宿主机上的 `~/lab/health-reports` 不受影响。

完成判定

□ 镜像构建成功并能跑出巡检输出　□ 报告通过挂载写入宿主机目录（容器删了报告还在）　□ 能说清"为什么容器里 systemd 检查会失败"以及你选了哪种处理办法　□ 能说出用 python:3.14-slim 当底座的好处

实战价值："容器里遇到的坑"——"容器里没有 systemd 导致服务检查失效，我改成端口探活/参数跳过"就是一段很实在的踩坑故事。

7-05 接到监控：把巡检指标暴露给 Prometheus

这一关让项目"活起来"：之前是"跑一次看一次"，现在是**持续暴露指标，让 Prometheus 每隔 15 秒来抓一次，超阈值还能触发告警。**这就是把脚本变成"可观测的服务"。

步骤 1：给脚本加一个 /metrics 出口（最简做法）

```
# 用标准库起一个极小的 HTTP 服务，把巡检结果按 Prometheus 格式吐出来
import re
from http.server import BaseHTTPRequestHandler, HTTPServer

LAST = {"disk": 0.0, "mem": 0.0, "load": 0.0}


def _num(text, pattern=r"([\d.]+)%"):
    """从检查函数的"人话"里抠出数字（巡检函数返回的是给人看的字符串）"""
    return float(re.search(pattern, text).group(1))


def refresh():
    """跑一次检查，把数字塞进 LAST"""
    LAST["disk"] = _num(check_disk(threshold=0)[1])                          # 根分区使用率 42% →
42.0
    LAST["mem"] = _num(check_memory(min_free_pct=0)[1], r"（([\d.]+)%）")  # 可用内存 …（78%）
→ 78.0
    LAST["load"] = _num(check_load(factor=0)[1], r"负载 ([\d.]+)")          # 1 分钟负载 0.31…
→ 0.31


class Metrics(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path != "/metrics":
            self.send_error(404); return
        refresh()
        body = (
            "# HELP health_disk_percent 根分区使用率\n"
            "# TYPE health_disk_percent gauge\n"
            f'health_disk_percent {LAST["disk"]}\n'
            f'health_mem_available_percent {LAST["mem"]}\n'
            f'health_load1 {LAST["load"]}\n'
        )
        self.send_response(200)
        self.send_header("Content-Type", "text/plain; version=0.0.4")
        self.end_headers()
        self.wfile.write(body.encode())


# —— 把上面这段（import re 起、到 Metrics 类为止）追加到 server_check.py，
#    再把 main() 的开头加上这个开关，--serve 才会起服务：
#     if "--serve" in sys.argv:
#         print("指标服务已启动：http://localhost:9101/metrics（Ctrl+C 退出）")
#         HTTPServer(("0.0.0.0", 9101), Metrics).serve_forever()
```

解读：**Prometheus 的指标格式其实很简单** —— 一行一个"指标名 值"，前面用 `# HELP / # TYPE` 声明。你不需要装 prometheus_client 库也能做（装了更省事：`pip install prometheus_client` 然后用 `Gauge`）。

更省事的写法（推荐）：`pip install prometheus_client`，用它自带的 `start_http_server(9101)` + `Gauge("health_disk_percent", "...").set(值)` —— 格式、并发、并发安全它都替你处理。

步骤 2：起服务并验证

```
$ python3 server_check.py --serve &
$ curl -s http://localhost:9101/metrics | head -6
# HELP health_disk_percent 根分区使用率
# TYPE health_disk_percent gauge
health_disk_percent 42.0
health_mem_available_percent 78.0
health_load1 0.31
```

步骤 3：让 Prometheus 抓它 + 配一条告警

```
$ cd ~/lab/monitor
# docker-compose.yml 的 prometheus 服务下加两行（Linux / WSL 原生 Docker 必须加，
# 否则 host.docker.internal 解析不了，Target 会一直 down）：
#   extra_hosts:
#     - "host.docker.internal:host-gateway"
# prometheus.yml 的 scrape_configs 里加一段：
  - job_name: "health-check"
    static_configs:
      - targets: ["host.docker.internal:9101"]
$ docker compose up -d prometheus
→ Status → Targets 里出现 health-check 且 UP

# alerts.yml 里加一条规则（照 5-05 的写法；注意整段要放在 groups → rules 下面）：
  groups:
  - name: health-check
    rules:
    - alert: HealthDiskHigh
      expr: health_disk_percent > 80
      for: 1m
      annotations:
        summary: "巡检脚本报告：磁盘使用率 {{ $value }}%"
```

解读：**到这里，你的脚本从"一段代码"变成了"系统的一部分"**：它自己暴露指标、被监控抓取、能触发告警。这就是"脚本 + 面板 + 告警"那条完整故事。

回滚方式

停掉服务 `kill %1`；把 prometheus.yml 里那段删掉并 `docker compose up -d`。

完成判定

□ `/metrics` 能返回至少三个自定义指标　□ Prometheus 的 Targets 里有 health-check 且 UP　□ Grafana 上能画出你自己的指标曲线　□ 有一条自己的告警规则，且调低阈值能触发　□ 能说出 Prometheus 的指标格式长什么样

实战价值：这一关把"我写过脚本"升级成"我做过可观测性"——别人听到"我自己暴露 metrics 接进 Prometheus，还配了告警"，会明显多看你一眼。

7-06 README 与设计决策记录

步骤 1：README（照 16 册那页的五段结构写）

```
# server-health-check

一台机器的健康巡检工具：磁盘 / 内存 / 负载 / 服务四项检查，超阈值告警，
可选容器运行，指标可被 Prometheus 抓取。

## 怎么用
    git clone https://github.com/你的用户名/server-health-check.git
    cd server-health-check && python3 server_check.py
    python3 server_check.py --serve        # 以 /metrics 方式常驻（端口 9101）

## 环境
Ubuntu 26.04 / Python 3.14；容器方式需要 Docker。

## 效果
    $ python3 server_check.py
    [正常] 磁盘：根分区使用率 42%
    [告警] 磁盘：根分区使用率 81%
    共 1 项异常

## 踩坑记录
- 容器里没有 systemd → 服务检查改成端口探活 / --skip-service
- 报告写进容器内部 → 一删容器就没了 → 改成挂载宿主目录
```

步骤 2：设计决策记录（最值得给人看的一页）

```
# 设计决策记录

1) 为什么用 subprocess 调系统命令，而不是读 /proc？
   选了：subprocess
   其它方案：直接读 /proc/meminfo 等文件
   为什么不用：df/free/uptime 的输出就是运维日常看到的格式，一致的输出方便对照；
               /proc 格式各发行版有差异，维护成本高

2) 为什么容器版把"服务检查"做成可选？
   选了：--skip-service ＋ 端口探活两种
   其它方案：容器里装 systemd
   为什么不用：容器里跑 systemd 是反模式（一个容器一个进程），得不偿失

3) 为什么报告按时间戳分文件，而不是一个文件追加？
   选了：每次一个文件
   其它方案：单文件追加
   为什么不用：单文件久了难检索，也容易被误删；分文件天然形成历史序列

4) 为什么自己暴露 metrics，而不是让脚本直接发告警通知？
   选了：暴露指标 ＋ Prometheus 告警规则
   其它方案：脚本内直接发邮件/钉钉
   为什么不用：脚本只负责"产出数据"，判断和通知交给监控系统，职责更清楚，也更容易改阈值
```

解读：**这四条的格式就是实战里最看重的那道题的答题格式**："我选了什么 / 还有什么方案 / 为什么不用"。写下来，讲的时候就不用现想。

完成判定

□ README 五段齐全，陌生人照它能跑通　□ 有设计决策记录，至少覆盖四个决策　□ 每条决策都写清"选了什么 / 还有什么 / 为什么不用"　□ 仓库里没有密钥等敏感信息

回滚方式：文档改动可随时回退（git 里都有）。

实战价值：这两份文档就是实战里最看重的那道题的"逐字稿"——你不需要临场组织语言，照着讲就行。

7-07 三分钟讲述稿与 10 分钟深挖

步骤 1：三分钟讲述稿（骨架照这里填，别背全文）

```
【1 分钟 · 是什么】
"这是我写的服务器健康巡检工具，检查四项：磁盘、内存、负载、关键服务。
 它可以在本机直接跑，也可以容器运行；指标能被 Prometheus 抓，超阈值会告警。"

【1 分钟 · 为什么这么做】
"我不想每次上机器手动敲四五个命令，就把它们写成脚本；
 判断交给 Prometheus，脚本只负责产出数据 —— 这样改阈值不用改代码。"

【1 分钟 · 踩了什么坑】
"容器版第一次跑就失败了：容器里没有 systemd，服务检查用不了。
 我改成两种方案：跳过参数，或者用端口探活代替。"

（结尾留一句钩子："如果您想看，我可以讲一下报告的历史对比是怎么做的。"）
```

步骤 2：准备 10 分钟深挖（一定会被追问）

可能的追问	你的答法（从项目里找证据）
为什么不用现成的监控？（Node Exporter 不就有了）	"现成的确实更全。我写它的目的是：① 练 subprocess + 异常 + 日志这条链；② 我加了业务侧的"关键服务是否活着"这种自定义判断 —— 这正是脚本比通用 exporter 灵活的地方。"
阈值为什么定 80%?	"80% 是我按经验定的保守值，真正的做法是先跑一周看基线；这也是我加了历史对比的原因。"
脚本挂了怎么办?	"分两层：脚本自身的异常有退出码 1，Prometheus 那边如果抓不到 targets 会显示 DOWN —— 两种情况都能被发现。"
要你巡检 100 台机器呢?	"本机版先解决单机；多机我会：先用 Ansible（批量在多台机器上执行命令的工具）/SSH 批量执行（这是我下一步想学的），或者每台部署一个 —— 指标层用 Prometheus 汇总就能看到全部。"
容器版为什么不用 host 网络?	"可以用，但会占用宿主端口；我选了挂载报告目录，指标用 host.docker.internal 暴露，避免和宿主服务冲突。"

步骤 3：对着人（或录音）讲一遍

```
要求：3 分钟讲述稿 不看稿；10 分钟深挖至少能接住 3 个追问
录音存进 learning-journey，文件名：server-health-check-讲述-YYYYMMDD.m4a
```

完成判定

□ 三分钟讲述稿能不看稿讲完（录音为证）　□ 能接住至少三个追问（照上面表里练）　□ 每个答案都能指到项目里的具体文件/命令　□ 讲述稿里至少说清一个"设计取舍"

回滚方式：无操作。

实战价值：实战里最看重的那道题的全部准备工作就在这一关里 —— 讲得清楚，比项目本身复杂更重要。

7-08 阶段 7 • 学习总结练习：陌生人 clone 下来能跑

业务场景

你要把这个工具交给别人用（别人、同事、或者三个月后的自己）。**一个自己不熟悉的人，能不能只看 README 就把它跑起来？**这一关就是验收这件事。

综合要求（这册的技能全用上）

技能点：**检查清单设计（7-01）+ subprocess 与阈值告警（7-02）+ 报告与历史对比（7-03）+ Dockerfile 与挂载（7-04）+ 指标暴露与 Prometheus 告警（7-05）+ README 与设计决策（7-06）+ 三分钟讲述（7-07）**

① 仓库里四件套齐全：`server_check.py`、`Dockerfile`、`README.md`、`设计决策记录.md`；
② 在**一台干净的机器**（或新建的 Ubuntu 虚拟机/WSL）上照 README 跑一遍；
③ 记录过程中"卡住的地方"，回头把 README 补清楚（这一步最容易发现问题）；
④ 跑通三件事：本机运行出报告 / 容器运行出报告 / `--serve` 后 Prometheus 抓得到；
⑤ 更新讲述稿：把这次"交接给别人"发现的坑加进去。

验收

□ 在干净环境照 README 跑通（截图或记录）　□ 三个跑法（本机 / 容器 / 指标）都验证过　□ README 里补上了"我卡住过的地方"　□ 设计决策记录至少四条　□ 三分钟讲述稿能不看稿讲完　□ 仓库里无密钥、无大文件

产出

仓库 `server-health-check`（README + 脚本 + Dockerfile + 设计决策记录 + 截图）。**这是你的头号项目。**

AI 运维指向

这套东西换成 GPU 机器就是 AI 集群巡检的雏形：把"磁盘/内存/负载/服务"换成"显存/GPU 利用率/推理服务是否活着"，其余骨架一模一样 —— 8-08 你已经做过 GPU 指标那一半了。

实战价值

实战里最看重的那道题的完整答案：有项目、有 README、有设计取舍、有踩坑记录、有讲述稿。**别人拿着你的仓库能跑起来 —— 这一条本身就是硬证据。**

顺带一提 • 版本 2（远程巡检）先不做

是什么：用 paramiko 通过 SSH 去巡检**别的机器**（现在这版只查本机）。
为什么留着：你入行后每天面对的是几十上百台机器，多机巡检是必然要做的；但现在做它的性价比低（先把单机版吃透更值）。
什么时候做：入行后有空时，照着 06 场景 5 的 paramiko 雏形往下写即可（那一关标着 ☆）。

《综合项目（server-health-check）》